数据结构核心考点期末复习讲义（第二册 · Modern C++ 重构版）

> **修订声明**：本文档基于原考点整理稿全面去噪重构。纠正了原稿中大量逻辑断层与拼写错别字；所有伪代码均已剥离裸 `malloc / free` 与 C 风格宏定义，规范化为现代 C++11 标准。
>
> **版权声明**：本文档整理与重构所有权归属 **© Akiro**。个人复习可用，未经授权禁止商用或二次打包传播。

导读：期末答题避坑指南（Akiro 批注）

1. **“头指针”与“头结点”的卷面混淆**：期末选择题极为热爱考察**空链表的判定条件**。进考场拿到卷子第一秒，先看清题目问的是“不带头结点的单链表”**（判定条件为 `head == nullptr`）还是**“带头结点的单链表”（判定条件为 `head->next == nullptr`）！
2. **算法与程序的本质区别**：程序可以无期限死循环执行（例如操作系统的事件监听内核），但**算法必须具备有穷性**（在有限步骤和时间内必须停机）。填空题挖空必考。

第 1 章 数据结构概述

1.1 基本概念与术语层次

- **数据 (Data)**：是对客观事物的符号表示，在计算机科学中是指所有能输入到计算机中并被计算机程序所处理的符号的总称。
- **数据元素 (Data Element)**：数据的**基本单位**，在集合中作为个体存在，也称之为元素、结点、顶点或记录。

  *（补充：一个数据元素可由若干个**数据项**组成。数据项是数据不可分割的最小单位）*
- **数据对象 (Data Object)**：具有**相同性质**的数据元素的集合，是数据的一个子集（在面向对象中常对应“属性”概念）。
- **数据结构 (Data Structure)**：相互之间存在一种或多种特定关系的数据元素的集合。

  *（数学概念抽象：**二元组**是一种用来表示某个数据对象以及各个元素之间关系的有限集合）*

1.2 逻辑结构与存储结构映射

Plaintext

```
数据的逻辑结构（独立于计算机内存）
├─ 集合结构 ：元素之间除“同属一个集合”外无其他关系
```

├─ 线性结构 ：元素之间存在一对一关系
├─ 树形结构 ：元素之间存在一对多关系（根除外，一前驱多后继）
└─ 图状结构 ：元素之间存在多对多关系

想要计算机处理数据，必须将逻辑结构映射为以下两种**物理存储结构**：

1. **顺序存储结构**：把逻辑上相邻的元素存储在物理内存也相邻的单元中，借助相对位置表达逻辑关系。
2. **链式存储结构**：借助指针表达逻辑关系，不要求物理位置相邻。

1.3 算法时空复杂度与评价标准

- **常见时间复杂度量级比较（卷面必考排序）**：

$$O(1) < O(\log_2 n) < O(n) < O(n\log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

- **算法五大特性**：① 输入、② 输出、③ **有穷性**、④ **确定性**（无歧义）、⑤ **可行性**。
- **算法设计五大要求**：① 正确性、② **健壮性**（对非法数据能妥善反应）、③ 可读性、④ 可修改可扩展性、⑤ 高效率。

第 2 章 线性表（重构实战专栏）

2.1 核心理论速记

- **端点边界**：线性表中，第一个元素没有直接前驱，最后一个元素没有直接后继。
- **指针删改**：单链表中若 `p` 是 `q` 的直接前驱，删除 `q` 的规范操作为：`p->next = q->next; delete q;`。
- **开销统计**：长度为 N 的顺序表，插入新元素平均移动 $\frac{N}{2}$ 个元素；删除元素平均移动 $\frac{N-1}{2}$ 个元素。
- **场景选型**：若线性表主要操作是在末尾增删，采用**顺序表**存储最节省时间。
- **地址求解**：设顺序表每个元素占 3 个单元，第 13 个元素（下标 12）地址为 336，则首地址 $LOC = 336 - 12 \times 3 = 300$。

2.2 带头结点单链表 C++ 面向对象重构

> 原稿 存在 `previos` 拼写错误及裸指针混用，标准工程化写法如下：

C++

```
#include <iostream>

typedef int DataType;
```

```
struct ListNode {
    DataType data;
    ListNode* next;
    ListNode(DataType val = 0) : data(val), next(nullptr) {}
};

class SinglyLinkedList {
private:
    ListNode* head;

    // 内部辅助函数：寻找第 k 个结点
    ListNode* getElem(int k) {
        if (k < 0) return nullptr;
        ListNode* p = head;
        for (int i = 0; p != nullptr && i < k; ++i) {
            p = p->next;
        }
        return p;
    }

public:
    SinglyLinkedList() {
        head = new ListNode(); // 初始化带头结点单链表
    }

    // 在第 i 个位置前插入元素 x (1-based 逻辑规约)
    bool insert(int i, DataType x) {
        ListNode* q = getElem(i - 1); // 找到第 i-1 个点
        if (q == nullptr) {
            std::cout << "非法插入位置!\n"; return false;
        }
        ListNode* s = new ListNode(x);
        s->next = q->next;
        q->next = s;
        return true;
    }

    // 删除第 i 个位置的元素
    bool remove(int i) {
        ListNode* q = getElem(i - 1);
        if (q == nullptr || q->next == nullptr) {
            std::cout << "非法删除位置!\n"; return false;
        }
        ListNode* p = q->next;
        q->next = p->next;
        delete p; // C++ 内存释放
        return true;
    }
```

```
    ~SinglyLinkedList() {
        ListNode* p = head;
        while (p != nullptr) {
            ListNode* tmp = p->next;
            delete p;
            p = tmp;
        }
    }
};
```

2.3 期末选择题高频考点大汇编（Akiro 归纳）

考点描述

题目设问核心

正确标准结论

出处溯源

链表空判定

不带头结点的单链表 `head` 为空条件

`head == nullptr` (原题选A)

链表空判定

带头结点的单链表 `head` 为空条件

`head->next == nullptr` (选B)

原子后插逻辑

在结点 `p` **之后**插入结点 `s`

`s->next = p->next; p->next = s;` (选B)

原子删除逻辑

删除结点 `p` 的**后续**结点

`p->next = p->next->next;` (选A)

有序表查找

n 个结点的有序单链表查待定元素

平均需比较 $\frac{n+1}{2}$ 个（约 $\frac{n}{2}$，选B）

时间复杂度

由 n 个向量建有序单链表

$O(n^2)$ *(注：原文档答案印成B有误，严谨应为 $O(n^2)$)*

时间复杂度

在已知结点后插 / 给定值后插

已知点后插为 $O(1)$；查值后插为 $O(n)$

问答大题：顺序表与链表的优缺点对比

- **顺序表**：存取速度快 $O(1)$，但空间一经定义不易扩充；且增删时平均需移动近一半元素，修改效率不高。
- **链表**：动态内存分配，**修改效率高**（只需修改指针，不必移动物理数据）。但只能循链顺序访问，**存取效率低** $O(n)$。
- **选型结论**：若表的总数基本稳定，很少增删，但要求**极速存取**，一律选用**顺序表**。

第 3 章 栈和队列

3.1 核心特性与判定大表

- **本质特征**：栈是**先进后出 (LIFO)**，队列是**先进先出 (FIFO)**。
- **运算效率**：无论采用顺序还是链式存储，栈与队列的增删操作时间复杂度永远是 $O(1)$。
- **出队 vs 取队头**：出队会彻底**删除**队头数据，取队头仅**获取值**而不删除。
- **链栈的核心优势**：相比顺序栈，链栈**永远不会出现栈满的情况**（除非系统内存耗尽，选择题选D）。
- **不可能出栈推演**：入栈序列为 `ABCDE`，绝对不可能出现的出栈序列是 `EDACB`（选择题选B）。

  (避坑验证：若E先出，说明ABCDE已全压入栈，此时出栈顺序只能被死死锁定为EDCBA，绝不可能让D出完跳过C直接出A）

3.2 链式队列与链栈 C++ 规范实现

考点 1：链式队列封装 (LinkQueue)

C++

```
struct QNode {
    int data;
    QNode* next;
    QNode(int val = 0) : data(val), next(nullptr) {}
};
```

```
class LinkedQueue {
private:
    QNode* front;
    QNode* rear;
public:
    LinkedQueue() {
        front = rear = new QNode(); // 附设头结点
    }

    void enQueue(int e) {
        QNode* p = new QNode(e);
        rear->next = p;
        rear = p;
    }

    bool deQueue(int& e) {
        if (front == rear) return false; // 队空
        QNode* p = front->next;
        e = p->data;
        front->next = p->next;

        if (rear == p) rear = front; // 若删的是队尾，尾指针回指向头结点
        delete p;
        return true;
    }
};
```

考点 2：链栈封装 (LinkedStack)

C++

```
struct StackNode {
    int data;
    StackNode* next;
    StackNode(int val = 0, StackNode* p = nullptr) : data(val), next(p) {}
};

class LinkedStack {
private:
    StackNode* top; // 链栈不需要头结点，top 直接指向有效栈顶
public:
    LinkedStack() : top(nullptr) {}

    bool isEmpty() const {
        return top == nullptr;
    }

    void push(int val) {
        top = new StackNode(val, top);
    }
```

```
    bool pop(int& val) {
        if (isEmpty()) return false;
        StackNode* p = top;
        val = p->data;
        top = top->next;
        delete p;
        return true;
    }
};
```

第 4 & 5 章 串、数组与广义表

- **串定义**：由零个或多个字符组成的有限序列。赋值写法：`string x = "abc";`。
- **多维数组考点**：务必在卷面上理清**按行存储**与**按列存储**的寻址定位换算。
- **稀疏矩阵存储选型**：① 三元组表、② 带辅助行向量二元组、③ **十字链表**。

十字链表矩阵结构重构

C++

```
struct OLNode {
    int row, col; // 行、列下标
    int value;    // 元素值
    OLNode *right, *down; // 行后继、列后继指针
};

struct CrossList {
    int mu, nu, tu; // 矩阵行数、列数、非零元个数
    OLNode** rhead; // 行表头指针数组首地址
    OLNode** chead; // 列表头指针数组首地址
};
```

第 6 章 树和二叉树

6.1 树与二叉树基本概念辨析

- **度**：结点的度 = 拥有的子树个数；树的度 = 树内各结点度数的**最大值**。
- **路线亲缘**：祖先与子孙靠连通路径界定。数据结构默认探讨**有序树**。
- **核心暴击点**：普通树只有前序、后序和层序遍历，**普通树绝对没有中序遍历**（因为子树不分左右）！

6.2 二叉树核心性质公式

1. **满二叉树**：叶子只能出现在最下一层，树中**只有度为 0 和度为 2 的结点**。
2. **完全二叉树**：叶子只能集中在**最下两层且靠左**。度为 1 的结点**最多只有 1 个，且必定为左孩子**。
3. **关键性质定理**：
   - 第 i 层最多 2^{i-1} 个结点。
   - 深度为 k 的二叉树最多 2^k-1 个结点。
   - **叶子公式**：$n_0=n_2+1$。
   - n 个结点的完全二叉树深度：$\lfloor \log_2 n \rfloor+1$。

6.3 二叉树经典大题演练（Akiro 细解）

例题 1：完全二叉树叶子求解

> **题目**：已知一棵完全二叉树有 47 个结点，求其叶子结点数。

标准解题推演：

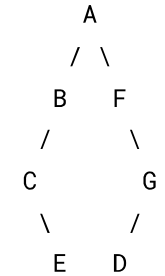
1. 深度推导：满 5 层总结点为 $2^5-1=31$，满 6 层为 63。故该树深度为 6 层。
2. 第 1 ~ 5 层满总结点数为 31。
3. 第 6 层（最底层）叶子数为：$47-31=16$ 个。
4. 第 6 层由 16 个孩子倒推，占用了第 5 层 $\frac{16}{2}=8$ 个双亲结点。
5. 第 5 层总共应有 $2^{5-1}=16$ 个结点，被占用 8 个，剩余叶子数为 $16-8=8$ 个。
6. 总体叶子数 = 第 6 层叶子 + 第 5 层叶子 = $16+8=\mathbf{24}$ **个**（原题选C）。

例题 2：唯一树结构还原

> **题目**：已知二叉树前序为 `ABCDEFG`，中序为 `CBEDAFG`，画出树形。

Plaintext

还原结构示意图（原题选C图）：



例题 3：极端的二叉树形态

- 前序 == 中序 $\Rightarrow$ 该树**只有右子树**；
- 中序 == 后序 $\Rightarrow$ 该树**只有左子树**；
- 前序 == 后序 $\Rightarrow$ 该树**只有一个根结点**或为空树。

二叉树先序遍历递归底层写法

C++

```
struct TreeNode {
    char data;
    TreeNode *lchild, *rchild;
};

void preOrder(TreeNode* root) {
    if (root != nullptr) {
        std::cout << root->data; // 先访问根
        preOrder(root->lchild);  // 递归左子树
        preOrder(root->rchild);  // 递归右子树
    }
}
```

第 7 ~ 10 章 图与查找表速览

7.1 图基本概念

- **完全图边数公式**：n 个顶点的无向完全图边数为 $\frac{1}{2}n(n-1)$；有向完全图弧数为 $n(n-1)$。
- **路径与连通**：起点与终点相同的路径称为**回路（环）**。图中任意两点存在路径即为**连通图**。
- **邻接矩阵逻辑表达**：

$$A[i][j] = \begin{cases} w_{i,j} & \langle v_i, v_j \rangle \in VR \\ 0 & \text{若边不存在} \end{cases}$$

- **核心遍历**：① **深度优先遍历 (DFS)**（仿树先序，利用标志位标记已访问）、② **广度优先遍历 (BFS)**（利用队列逐层推进）。

7.2 查找与排序

- **静态查找表**：仅作查询与检索属性操作，表结构运行期固化。
- **动态查找表**：查找过程中伴随动态增删结点。
- **排序本质**：将任意序列重新排列成一个**按关键字有序**的序列。